# EduPlatform — Phase 0 Report

Project foundation: what was built and how

Diploma thesis project · Branch **feat/phase-0-project-skeleton** · 19 tests passing · verified running on localhost · 21 August 2026

Phase 0 delivers no end-user feature by design. Its goal is that every following day of work starts with a running environment instead of setup: one command brings up the infrastructure, one command starts the API, one command starts the client, and the test suite and CI pipeline are already in place to catch regressions from the first feature onward.

## 1. Local infrastructure

Five containers, started with a single `docker compose up -d`.

| What | How it was done |
|---|---|
| **PostgreSQL 17 + pgvector** | Uses the `pgvector/pgvector:pg17` image so the vector extension needed in Phase 8 is already present. An init script enables `vector`, `pg_trgm` and `unaccent` on first start, and the database is created with Bulgarian ICU collation so sorting behaves identically on every machine. |
| **Redis** | Started with append-only persistence enabled so cached data survives a container restart. It will serve as cache, session store and SignalR backplane from Phase 7. |
| **MinIO + bucket bootstrap** | S3-compatible object storage for materials and homework submissions. A one-shot `minio-init` job runs the MinIO client to create the four buckets (materials, submissions, avatars, exports) and then exits. |
| **Mailpit and Seq** | Mailpit captures outgoing e-mail locally so nothing is ever sent to a real address during development. Seq collects structured logs from the API and makes them searchable in a browser. |
| **Embeddings container (deferred)** | The bge-m3 embedding server is declared behind a Docker Compose profile named `ai`, so it is not pulled by default — it downloads roughly 2 GB and is only needed from Phase 8. |
| **Health checks** | Every service declares a healthcheck, so dependent containers wait for readiness rather than starting against a database that is not accepting connections yet. |

## 2. Solution structure

Eleven .NET projects, laid out to enforce the modular-monolith boundaries described in the architecture document.

| What | How it was done |
|---|---|
| **Project layout** | Created an `EduPlatform.slnx` solution holding four BuildingBlocks projects, four projects for the Identity module (Domain, Application, Infrastructure, Contracts), the API host, and two test projects. |

| What | How it was done |
|---|---|
| **Dependency direction** | Project references were wired so that dependencies point inward only: Infrastructure depends on Application, Application on Domain, and Domain on nothing. A module may reference another module's Contracts project, never its internals. |
| **Shared build settings** | A single `Directory.Build.props` sets the target framework, nullable reference types and analysis level for every project at once, so no individual project file repeats them. |
| **Central package management** | `Directory.Packages.props` declares each NuGet version exactly once. Individual project files reference packages without a version, which makes conflicting versions across projects impossible. |
| **Strict builds in CI only** | Warnings are treated as errors when the `ContinuousIntegrationBuild` flag is set. The pipeline stays strict without interrupting local work mid-edit. |

## 3. Domain building blocks

The base types every future entity in the system will inherit from.

| What | How it was done |
|---|---|
| **Entity and AggregateRoot** | `Entity` gives every domain object a typed identity and identity-based equality, so two instances loaded separately compare as equal when their ids match. `AggregateRoot` adds a private list of raised domain events with a read-only view over it. |
| **ValueObject** | An abstract base implementing structural equality from a component list, so objects like a grade value or a date range compare by content instead of by reference. |
| **Domain events** | `IDomainEvent` marks an event; aggregates raise them during business operations rather than calling other modules directly. This is what will let Phase 7 add notifications without touching Phases 4–6. |
| **DomainException** | A dedicated exception type for broken business rules, which the API translates into a 400 response while genuine bugs still surface as 500. |

## 4. CQRS core

A hand-written command and query dispatcher, roughly 150 lines, replacing MediatR.

| What | How it was done |
|---|---|
| **Dispatcher** | Resolves the handler for a given command or query from the dependency-injection container and invokes it. Handlers are discovered and registered by assembly scanning, so adding a feature requires no registration code. |
| **Pipeline behaviours** | Cross-cutting concerns wrap each handler as a chain, in the same way ASP.NET Core middleware wraps a request. |

| What | How it was done |
|---|---|
| **Validation behaviour** | Runs every FluentValidation validator registered for the incoming message before the handler executes, and throws with the collected field errors if any fail. No handler needs to validate its own input. |
| **Logging behaviour** | Logs the name and duration of each handled message, and flags any that exceeds 500 ms as a warning — an early indicator of a slow query. |
| **Clock abstraction** | `IClock` hides `DateTimeOffset.UtcNow` behind an interface so time-dependent logic, notably the Phase 6 test timer, can be tested deterministically. |

## 5. Persistence

Shared EF Core plumbing that each module will build its own database context on.

| What | How it was done |
|---|---|
| **ModuleDbContext** | An abstract base context that pins each module to its own PostgreSQL schema and applies that module's entity configurations automatically. |
| **snake_case naming** | The EFCore.NamingConventions package converts C# PascalCase names into PostgreSQL snake_case, so the generated schema reads naturally when queried by hand. |
| **Domain event interceptor** | A save-changes interceptor collects the events raised by aggregates and publishes them *after* the transaction commits. This guarantees no side effect can fire for a change that was rolled back. |
| **Seeding mechanism** | An `IDataSeeder` interface plus a runner that resolves every registered implementation and executes them in declared order. It is triggered only by launching the host with `--seed`, which populates data and exits without serving traffic, so a deployment can never overwrite real data by accident. |

## 6. API host

The ASP.NET Core application that wires everything together.

| What | How it was done |
|---|---|
| **Serilog logging** | Structured logging writes to the console during development and to Seq over the network, with the environment name and correlation id attached to every entry. |
| **Correlation id** | Middleware assigns each request an id, echoes it in the response headers and attaches it to the log scope, so all log lines for one request can be filtered together in Seq. |
| **Problem Details errors** | A global exception handler converts unhandled exceptions into RFC 9457 responses. Validation failures become 400 with a per-field list, domain rule violations 400, and everything else a 500 that hides internals but carries a trace id. |

| What | How it was done |
|---|---|
| **OpenAPI and Scalar** | The API description is generated at build time and served through Scalar, giving a browsable, try-it-out documentation page at `/scalar`. |
| **Health probes** | `/health/live` reports whether the process is up; `/health/ready` additionally checks PostgreSQL, Redis and MinIO. These are the exact endpoints Kubernetes will use in Phase 9. |
| **CORS and configuration** | Allowed client origins are read from strongly typed options bound to configuration, rather than being hard-coded in the startup file. |
| **System endpoints** | A small `/api/v1/system/info` endpoint reports version, environment and dependency status. It exists to prove the whole chain — browser to API to database — works end to end. |

## 7. Angular client

Angular 22, generated with the CLI and then trimmed to the project's conventions.

| What | How it was done |
|---|---|
| **Zoneless change detection** | The application runs without zone.js, using Angular's signal-based change detection. This is the current default direction and avoids retrofitting later. |
| **Standalone components and lazy routes** | There are no NgModules; feature routes load their components on demand, which keeps the initial bundle small as features are added. |
| **Development proxy** | `proxy.conf.json` forwards `/api` calls from port 4200 to the API, so the browser sees a single origin and CORS never interferes during development. |
| **System status screen** | A typed service calls the system endpoint and the screen renders one indicator per dependency. This is the visible proof that Phase 0 works, and the first thing to check if the environment breaks. |

## 8. Tests

Nineteen tests, all passing, establishing the levels the project will keep using.

| What | How it was done |
|---|---|
| **Architecture tests — 5** | NetArchTest rules assert the module boundaries mechanically: the domain layer references no infrastructure, and no module reaches into another module's internals. A boundary violation now fails the build instead of being noticed at review. |
| **API pipeline tests — 5** | `WebApplicationFactory` starts the real application in memory and drives it over HTTP, covering routing, correlation id propagation, Problem Details on an unknown route, and the liveness probe. These touch no database and need no Docker. |

| What | How it was done |
|---|---|
| **Database tests — 4** | Testcontainers starts a real PostgreSQL 17 server from the same `pgvector/pgvector:pg17` image Docker Compose uses, then verifies connectivity, the pgvector cosine-distance operator, the trigram and unaccent extensions, and that two modules can hold same-named tables in separate schemas. A running Docker daemon is required. |
| **Seeding tests — 4** | Prove the seeding mechanism itself: seeders are discovered through dependency injection, run in their declared order, and an empty registration is a valid state rather than a crash. |
| **Frontend test — 1** | Runs on Vitest with jsdom and confirms the application bootstraps. No browser is needed. |

## 9. Continuous integration

A GitHub Actions workflow that runs on every push and pull request.

| What | How it was done |
|---|---|
| **Backend job** | Restores and builds the solution in Release with warnings promoted to errors, then runs the .NET test suite with coverage collection. The Ubuntu runner provides the Docker daemon the database tests need. |
| **Frontend job** | Installs from the lockfile, lints, runs the Vitest suite headless and produces a production build. |
| **Dependency audit** | A third job fails the build on a vulnerable NuGet package, including transitive ones, and runs `npm audit` at high severity for the client. Low-severity advisories in build-only packages are tolerated deliberately, since they never reach the browser. |

## 10. Deviations from the architecture document

Four choices made during implementation differ from the plan. Each is deliberate and worth mentioning at the defense.

| What | How it was done |
|---|---|
| **.NET 10 instead of .NET 9** | The installed SDK is 10.0.201, and .NET 9 is a Short Term Support release that has already reached end of support. .NET 10 is the current Long Term Support version and covers the full thesis timeline. |
| **No MediatR** | MediatR moved to the Reciprocal Public License, which would oblige anyone deploying the platform to publish their source. A ~150-line dispatcher replaces it, removes the licensing constraint, and is easier to explain in the thesis than a third-party library. |
| **Shouldly instead of FluentAssertions** | FluentAssertions version 8 and later require a paid licence for non-open-source use. Shouldly is functionally equivalent and unrestricted. |

| What | How it was done |
|---|---|
| **PostgreSQL on port 5433** | Port 5432 is already taken by a PostgreSQL service installed on the machine. The container is mapped to 5433 and the port is configurable through the environment file. |

## 11. How to verify

From the repository root, in order. The first command needs to run only once per session.

| Command | Result |
|---|---|
| `docker compose up -d` | Starts the five infrastructure containers. Wait for `docker compose ps` to report all healthy. |
| `dotnet run --project src/EduPlatform.Api` | Starts the API on ports 5000 (http) and 5001 (https). There is no migration step yet — no module defines entities before Phase 1. |
| `npm --prefix src/web start` | Starts the Angular client on port 4200, proxying /api to the API. |
| `dotnet run --project src/EduPlatform.Api -- --seed` | Populates development data and exits. Reports that no seeders are registered until Phase 1. |
| `dotnet test EduPlatform.slnx` | Runs all 18 .NET tests. The database tests need the Docker daemon running. |
| `npm --prefix src/web test -- --watch=false` | Runs the frontend test. |

| Address | Shows |
|---|---|
| `http://localhost:4200` | Angular application, system status screen |
| `http://localhost:5000/scalar` | API documentation, browsable |
| `http://localhost:5000/health/ready` | Dependency health as JSON |
| `http://localhost:5341` | Seq, structured logs |
| `http://localhost:9001` | MinIO console (minioadmin / minioadmin) |

**Not done in Phase 0, by design.** There is no seed *data* — no schools, classes or students — because the entities it would populate are created in Phases 1 and 2. The seeding *mechanism* is built and tested, so each module plugs into it by registering one class. There are likewise no EF Core migrations yet, for the same reason: no module defines an entity. The Identity module exists as four empty projects that establish the structure; its contents are Phase 1.

**Next: Phase 1 — Authentication and roles.** ASP.NET Core Identity on PostgreSQL, JWT access tokens with rotating refresh tokens, the four roles (Student, Teacher, Parent, Admin), and route guards in Angular so each role lands on a different screen.